

SecureWebApp

Secure Document Sharing System

CS 419: Computer Security | Spring 2026

Krystian Bochenko & Ethan Nayak

- Python / Flask
- bcrypt + Fernet AES-128
- RBAC + Audit Logging
- TLS / HTTPS



Project Overview

SecureWebApp is a Flask web application that lets users register, log in, upload encrypted documents, and share them with other users at different permission levels. Every file is encrypted before it touches the disk. Every sensitive action is logged.

User Auth

- Registration with bcrypt hashing
- Password complexity enforcement
- Account lockout after 5 fails
- Rate limited registration

Document Storage

- Fernet AES-128 encryption at rest
- Extension and MIME type checks
- Secure filename handling
- Version history per document

Sharing and RBAC

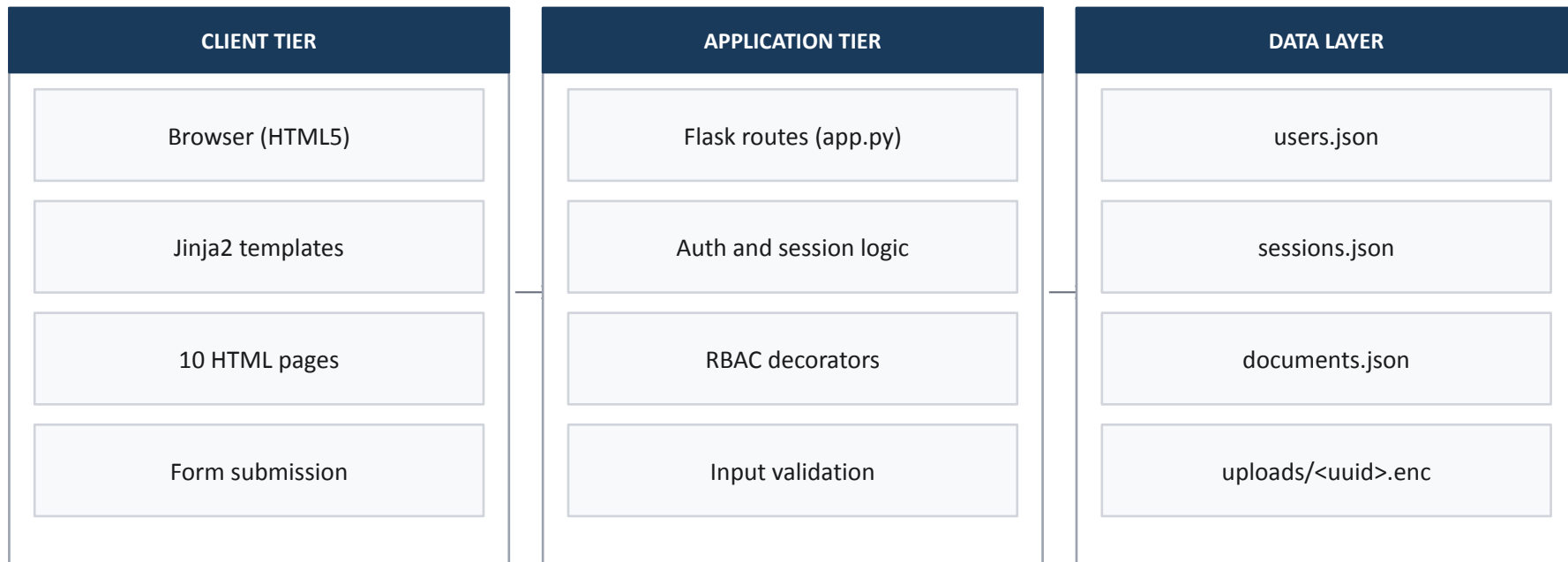
- Owner, Editor, Viewer roles
- Admin, User, Guest account types
- Per-document share controls
- Server-side auth on every route

Audit Logging

- JSON-structured security events
- 17 tracked event types
- Severity levels: INFO to ERROR
- Tested with real user sessions

Stack: Python / Flask · bcrypt · Fernet (cryptography) · flask-limiter · Werkzeug · JSON files · TLS

System Architecture



Security spans every tier: TLS in transit · bcrypt and Fernet at rest · RBAC decorators in the app layer · Every event is logged.

No external database — file-based JSON eliminates SQL injection entirely.

All routes run through `@require_auth` and `@require_role` before any business logic executes.

Authentication and Session Management

User Authentication

Password Hashing

bcrypt with cost factor 12. Each hash takes roughly 300ms, making offline cracking slow. Passwords are salted automatically and never stored in any readable form.

Complexity Rules

Minimum 12 characters. Must include uppercase, lowercase, a digit, and a special character. All five checks run as individual regex passes with specific error messages.

Account Lockout

After 5 failed logins the account is locked for 15 minutes. The lockout timestamp is written to users.json and logged as ACCOUNT_LOCKED at ERROR severity.

Rate Limiting

flask-limiter applies a 5 request per minute cap on /register to block mass account creation. /login has a 10 request per minute cap applied after remediation.

Session Management

Token Generation

secrets.token_urlsafe(32) produces 256 bits of cryptographically secure randomness — the standard library recommended method for session tokens.

Timeout

Sessions expire after 30 minutes of inactivity. validate_session() checks last_activity on every request and deletes expired sessions from sessions.json automatically.

Cookie Security

The session cookie is set with HttpOnly=True so JavaScript cannot read it, SameSite=Strict to block cross-site request forgery, and secure=True for HTTPS-only transmission.

Lifecycle Logging

SESSION_CREATED, SESSION_DESTROYED, and SESSION_EXPIRED events are all written to the security log. Confirmed working across multiple real test sessions.

Access Control and Input Validation

Role-Based Access Control

Feature	Admin	User	Guest
Register / Login	Yes	Yes	Yes
Upload documents	Yes	Yes	No
Download own docs	Yes	Yes	No
Download shared docs	Yes	Yes	Yes
Edit / delete own docs	Yes	Yes	No
View all documents	Yes	No	No
Manage users / roles	Yes	No	No

@require_auth and @require_role decorators wrap every protected route. Authorization failures return 403 and log an ACCESS_DENIED warning.

Input Validation

XSS Prevention

html.escape() runs on every form input before use. Jinja2 auto-escaping handles all template output. CSP header blocks scripts from external sources as a third layer.

File Upload Checks

Three independent checks on every upload: file extension against a whitelist, MIME type against a separate set, and secure_filename() to strip path separators and null bytes.

Field Validation

Username: 3 to 20 alphanumeric characters via regex. Email: format check with 254-character cap. Password: five separate regex checks, each returning a specific error message.

No Shell Commands

No os.system() or subprocess calls anywhere in the codebase. All file access uses Python's open() with paths built from UUIDs, not user input. Path injection is not possible.

Encryption and Audit Logging

Encryption

Files at Rest (Fernet)

Every uploaded file is encrypted with Fernet before being written to disk. Fernet uses AES-128-CBC for encryption and HMAC-SHA256 for integrity. Files are stored as <uuid>.enc and decrypted in memory only on an authorized download. If the HMAC check fails, decryption throws an error rather than returning bad data.

In Transit (TLS)

Flask runs with ssl_context pointing to a certificate and key file. This enables HTTPS on port 5000 so all traffic, including session cookies and file contents, is encrypted in transit.

Password Hashing (bcrypt)

Passwords are hashed with bcrypt, a one-way function. The original password cannot be recovered from the stored hash. bcrypt.checkpw() uses constant-time comparison to prevent timing attacks.

Security Audit Logging

How It Works

The SecurityLogger class wraps Python's logging module. Every call to log_event() writes a JSON object with a UTC timestamp, event type, user ID, and details dictionary. Severity maps to Python log levels: INFO for normal events, WARNING for denials, and ERROR for lockouts.

Events Logged

Auth	LOGIN_SUCCESS, LOGIN_FAILED, ACCOUNT_LOCKED, LOGIN_BLOCKED
Sessions	SESSION_CREATED, SESSION_DESTROYED, SESSION_EXPIRED
Access	ACCESS_DENIED, VALIDATION_FAILED
Documents	DOC_UPLOADED, DOWNLOADED, SHARED, EDITED, DELETED
Admin	USER_DELETED, ROLE_CHANGED, USER_REGISTERED

Verified by live test log: real lockouts, access denials, and role changes all captured.

Penetration Testing

Self-assessment using the OWASP WSTG methodology and project checklist

0

Critical / High

1

Medium

2

Low

3

Informational

MED

Debug Mode Enabled

Flask runs with `debug=True`, exposing the Werkzeug debugger on error pages. In production this allows arbitrary code execution and full stack trace exposure.

Fix: Set `debug=False` or use `os.environ.get('FLASK_DEBUG', False)`.

LOW

Self-Signed TLS Certificate

The app uses a self-signed cert. Browsers warn users the connection is not trusted, which may condition them to ignore certificate warnings over time.

Fix: Use a CA-signed certificate (e.g., Let's Encrypt) for any production deployment.

LOW

No Password Reset

No password reset feature exists. Users cannot recover accounts without direct admin help. Low security risk but affects usability and does not scale.

Fix: Add a reset flow using time-limited tokens (`secrets.token_urlsafe(32)`) with a 15-min expiration.

INFO

Concurrent Sessions Allowed

Multiple active sessions per user are permitted with no single-session enforcement or session listing UI.

Fix: Noted for awareness. Add session management UI or single-session enforcement if required.

INFO

Deprecation Warnings in Tests

Pytest surfaced 6 deprecation warnings. Primary issue is `datetime.datetime.utcnow()` in `security_logger.py`, deprecated in Python 3.14.

Fix: Replace with `datetime.datetime.now(datetime.UTC)` to resolve.

INFO

Missing python-dotenv

Flask detected a `.env` file but `python-dotenv` is not in `requirements.txt`, requiring manual environment variable export each session.

Fix: Add `python-dotenv` to `requirements.txt` to enable automatic `.env` loading.

Challenges and Lessons Learned

Security requires layered thinking

Every feature had to be considered from three angles at once: validation, authorization, and logging. A file upload needs an extension check, a MIME check, a size limit, a session check, an ownership record, and a log entry. Getting one wrong breaks the whole guarantee.

Key management is harder than encryption

Adding Fernet encryption was straightforward. Deciding how to store the key, keep it out of git, and what happens if it is lost took more design thought than the actual crypto. A production system would need versioned keys and a rotation plan.

Building the threat model first paid off

We wrote the STRIDE analysis before testing. When it came time to run the pen test checklist, the findings we identified as risks were the same ones that showed up. That made the report more honest and more useful.

File-based storage has real tradeoffs

Using JSON files means there is no SQL injection surface at all. But there are no atomic transactions either. A crash mid-write could leave the data in a bad state. We handled this by always building a complete object in memory before calling `save_json()`.

Logging is only useful if you read it

We ran real user sessions during development and verified the log output for each one. Seeing the `ACCOUNT_LOCKED` events appear exactly when expected and `ACCESS_DENIED` events fire correctly made it clear the security controls were working end to end.



Thank You!

CS 419: Computer Security | Spring 2026 | Rutgers University

Final Deliverables:

- Full Flask web app with user auth, role-based document sharing, and version history
- Fernet AES-128 encryption at rest and TLS in transit for all file and session data
- JSON-structured audit log covering 17 event types across the full application lifecycle
- Self-assessed pen test: 0 Critical/High, 1 Medium, 2 Low, 3 Informational findings (Fixes done/planned for production)

Closing notes:

In the end, after building the secure document sharing system from the ground up using Flask, bcrypt, and Fernet encryption. We learned that every design decision had to planned to have security in mind before even starting. Along the way we learned that security is not a feature you add at the end, it is something you build into every part of the system from the start. Despite even having some difficulties and facing steep learning curves, we both felt we took away several important skills and relevant experience that can be very applicable in our future of secure building in our time here. Thank you so much for taking the time to look over our project, we hope you had as much fun checking iit out as we did working on it!